

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Florian Adamczyk^a^aJustus-Liebig-Universität Gießen, Matrikelnr. 8105234

Gruppe 308 – KI I Projekt WS 2025/26

Zusammenfassung—Diese Zusammenfassung dokumentiert meinen individuellen Arbeitsprozess im Rahmen des KI I-Projekts zur Vorhersage kalifornischer Hauspreise mittels neuronaler Netze. Der Schwerpunkt liegt auf der Projektinfrastruktur, der explorativen Datenanalyse sowie der Entwicklung und Evaluation eines TensorFlow/Keras-basierten Regressionsmodells. Das beste neuronale Netz erreicht einen R^2 -Wert von 0,78 auf dem Testset und übertrifft damit die lineare Regression ($R^2 = 0,63$) deutlich.

Keywords—Neuronales Netz, California Housing, TensorFlow, Regression, Machine Learning

1. FRAGESTELLUNG UND ZIEL

Ziel des Projekts war die Entwicklung eines neuronalen Netzes zur Vorhersage von Immobilienpreisen auf Basis des California-Housing-Datensatzes. Gemäß Aufgabenstellung sollte das Modell mit einer linearen Regression verglichen und die Ergebnisse kritisch eingeordnet werden.

Meine zentrale Fragestellung lautete: *Kann ein neuronales Netz die nichtlinearen Zusammenhänge im Datensatz besser erfassen als eine lineare Regression, und welche Architektur eignet sich dafür?*

Darüber hinaus habe ich als Vorbereitung die gesamte Projektinfrastruktur (Repository, Code-Module, Notebooks) aufgebaut, um allen Teammitgliedern eine einheitliche Arbeitsgrundlage zu bieten.

2. METHODIK UND VORGEHEN

2.1. Projektinfrastruktur

Zu Beginn habe ich ein GitHub-Repository eingerichtet und nbstripout als Git-Hook konfiguriert, um Merge-Konflikte durch Notebook-Outputs zu vermeiden [1]. Die Dokumentation in Resources.md ermöglicht allen Teammitgliedern eine einheitliche Einrichtung.

Ein zentrales utils/-Modul stellt gemeinsame Funktionen bereit:

- **data.py:** Laden und Bereinigung des Datensatzes (Cut-offs bei gecappten Werten, Ausreißer-Filterung über 98. Perzentil)
- **evaluation.py:** Einheitliche Metriken (R^2 , MAE, RMSE) und Ergebnissammlung
- **plotting.py:** Standardisierte Visualisierungen mit Export nach results/

2.2. Explorative Datenanalyse

Der Rohdatensatz umfasst 20.640 Samples mit 8 Features. Nach der Bereinigung (Entfernung gecappter Werte und Ausreißer) verbleiben ca. 17.386 Samples. Fig. 1 zeigt die Korrelationsmatrix: Das mediane Einkommen (MedInc) korreliert am stärksten mit dem Hauspreis ($r \approx 0,69$).

2.3. Baseline: Lineare Regression

Als Referenzmodell dient eine lineare Regression auf allen 8 Features. Die Ergebnisse (Table 1) zeigen systematische Residuenmuster (Fig. 2), die auf nichtlineare Zusammenhänge hindeuten, welche die lineare Regression nicht erfassen kann.

2.4. Neuronales Netz mit TensorFlow/Keras

Für das neuronale Netz habe ich systematisch verschiedene Architekturen getestet:

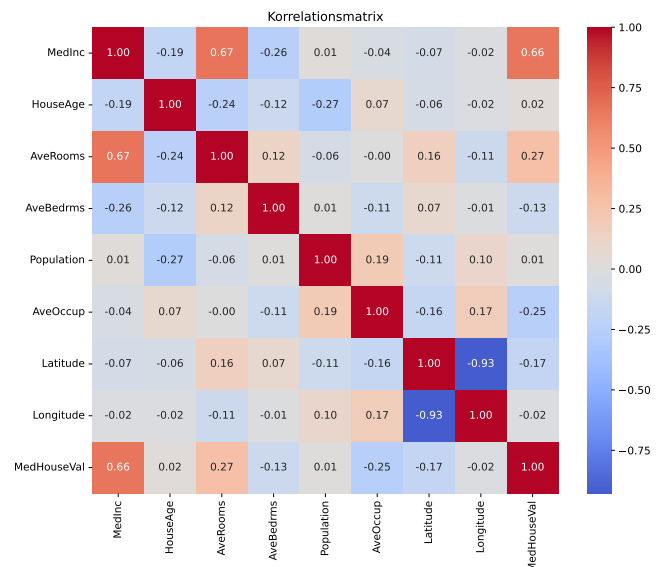


Abbildung 1. Korrelationsmatrix der bereinigten Features. MedInc zeigt die stärkste Korrelation zur Zielvariablen MedHouseVal.

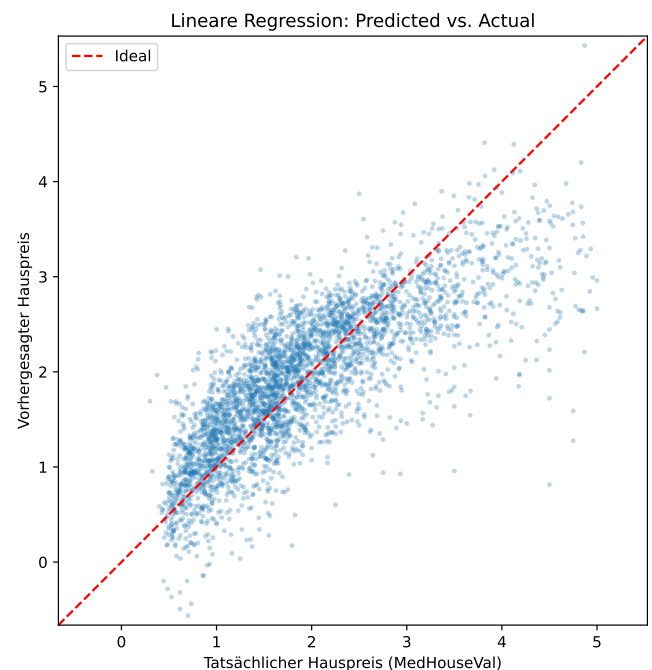


Abbildung 2. Predicted vs. Actual für die lineare Regression. Die Streuung um die Diagonale und systematische Abweichungen bei hohen Preisen zeigen die Grenzen des linearen Modells.

1. **Modell 1:** [32] – minimales Netz als Ausgangspunkt
2. **Modell 2:** [64, 32, 16] – tiefere Architektur
3. **Modell 3:** [128, 64, 32] – breiteres Netz
4. **Modell 4:** Vergleich verschiedener Aktivierungsfunktionen (ReLU, ELU, tanh, sigmoid)
5. **Modell 5:** [128, 64, 32, 16] + Dropout – Regularisierung gegen Overfitting

Alle Modelle wurden mit dem Adam-Optimizer (Lernrate 0,001) trainiert. Die Eingabedaten wurden standardskaliert, da neuronale Netze skalierte Features für effizientes Training benötigen. Fig. 3 zeigt die Lernkurven ausgewählter Modelle.

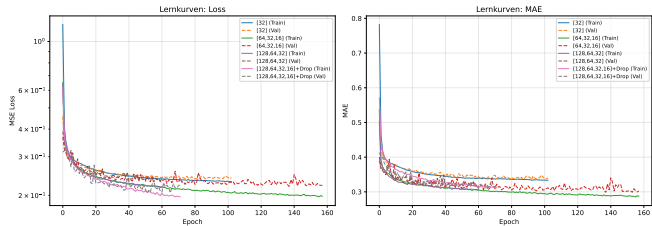


Abbildung 3. Lernkurven (Loss und MAE) für verschiedene NN-Architekturen. Das Modell mit Dropout zeigt die stabilste Generalisierung.

3. ERGEBNISSE

Table 1 fasst die zentralen Ergebnisse zusammen. Das beste neuronale Netz (Modell 5: [128, 64, 32, 16] + Dropout) erreicht einen R^2 -Wert von **0,78** auf dem Testset – eine Verbesserung um 15 Prozentpunkte gegenüber der linearen Regression.

Tabelle 1. Vergleich der Modellperformance auf dem Testset

Modell	R^2 Test	MAE	RMSE
Lineare Regression	0,633	0,434	0,578
NN [32]	0,712	0,362	0,512
NN [64, 32, 16]	0,748	0,331	0,479
NN [128, 64, 32]	0,726	0,345	0,499
NN + Dropout	0,780	0,307	0,448

MAE und RMSE in Einheiten von \$100.000. Das NN mit Dropout zeigt die beste Generalisierung.

Fig. 4 visualisiert den direkten Vergleich: Das neuronale Netz zeigt eine engere Streuung um die Diagonale, insbesondere im mittleren Preissegment.

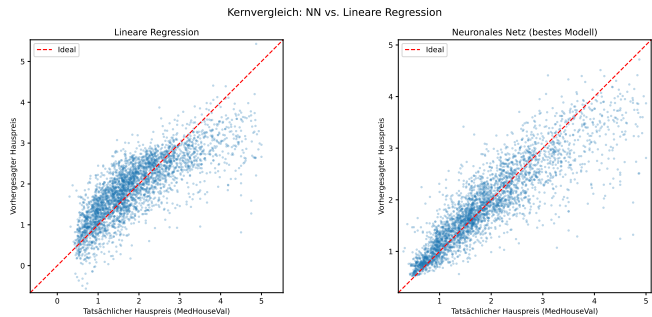


Abbildung 4. Kernvergleich: Lineare Regression (links) vs. bestes NN (rechts). Das NN approximiert die Diagonale deutlich besser.

Zentrale Erkenntnisse:

- Das NN erfasst nichtlineare Interaktionen zwischen Features (z. B. Einkommen $\times$ Lage), die der linearen Regression verborgen bleiben.

- Ohne Regularisierung (Modell 3) tritt Overfitting auf: $R^2_{\text{Train}} = 0,88$ vs. $R^2_{\text{Test}} = 0,73$.
- Dropout (20 % in den ersten Schichten) reduziert die Generalisierungslücke effektiv.
- ReLU-Aktivierung liefert konsistent die besten Ergebnisse.

4. SCHWIERIGKEITEN UND GRENZEN

4.1. Technische Herausforderungen

Python-Versionskonflikt: TensorFlow ≥ 2.17 unterstützt Python 3.14 nicht. Lösung: Wechsel auf Python 3.13. Für Teammitglieder mit Intel-Mac (TensorFlow-Inkompatibilität ab 2.17) wurde Python 3.11 mit TensorFlow 2.16 als Alternative dokumentiert.

Git-Workflow: Mehrere Teammitglieder hatten Schwierigkeiten mit Git-Befehlen. Ich habe individuellen Support via Screensharing geleistet und nbstripout auf allen Systemen eingerichtet.

4.2. Methodische Grenzen

- **Interpretierbarkeit:** Das NN ist eine Black Box – für regulierte Domänen (Kreditvergabe, Medizin) bleibt lineare Regression oft bevorzugt.
- **Hyperparameter-Tuning:** Architektur, Lernrate, Dropout-Rate, Epochenzahl – der Suchraum ist groß. Ein systematisches Grid-/Random-Search wurde noch nicht durchgeführt.
- **Rechenzeit:** Training auf CPU dauert ca. 1–2 Minuten pro Modell (300 Epochen). GPU-Beschleunigung wäre für umfangreicheres Tuning sinnvoll.

5. ZWISCHENSTAND UND AUSBLICK

Nach ca. 61 Stunden Arbeitszeit habe ich folgende Meilensteine erreicht:

1. Vollständige Projektinfrastruktur (Repository, Utils-Modul, 8 Notebooks)
2. Explorative Datenanalyse und Baseline-Modell
3. Systematischer Vergleich von 5 NN-Architekturen
4. Git-Support für das gesamte Team

Offene nächste Schritte:

- Early Stopping zur automatischen Epochenwahl
- Learning-Rate-Scheduling für feineres Tuning
- Vergleich mit Ensemble-Methoden (Random Forest, Gradient Boosting) aus den Notebooks anderer Teammitglieder
- Feature-Engineering (Polynomiale Features, Interaktionsterme)

Fazit: Das neuronale Netz übertrifft die lineare Regression deutlich und demonstriert die Fähigkeit, nichtlineare Muster im California-Housing-Datensatz zu lernen. Die Regularisierung durch Dropout ist essenziell für eine gute Generalisierung. Der erreichte R^2 -Wert von 0,78 stellt einen soliden Zwischenstand dar, der durch weiteres Hyperparameter-Tuning noch verbessert werden könnte.

LITERATUR

- [1] F. Rathgeber, *nbstripout – Strip output from Jupyter and IPython notebooks*, Zugriff: 11.02.2026, 2024. Adresse: <https://github.com/kynan/nbstripout>.